Λ-BRIEF · REVISÃO DO CÓDIGO LEGADO

Por que o λ-BRIEF ganhava com folga no artigo

O abBRIEF e o λ-BRIEF não usavam o mesmo baseline, e as 30 repetições não eram independentes. Onde isso está no código legado e como passamos a fazer.

Vitor Ochoa

Phi-Robotics · UFRGS · outubro de 2026

`phir2frameworkv2/` → `lambda_brief_modern/`

RESUMO

O ganho do artigo misturava o efeito da vegetação com diferenças de implementação

BASELINE

abBRIEF e λ-BRIEF rodavam por caminhos de código diferentes

O abBRIEF usava 256 pares fixos e quase uniformes. O λ usava pares no centro, guiados pelo NDVI e re-sorteados a cada frame.

REPETIÇÕES

As 30 repetições eram uma cadeia, não 30 testes

A partir da 2ª repetição a nuvem inicial era sempre a mesma e a escala z vinha do fim da repetição anterior.

EFEITO

A vantagem real do λ é convergir mais vezes

No motor novo, com o mesmo baseline e seeds independentes, o λ converge em mais execuções. Quando os dois convergem, o erro é igual.

COMO O DESCRITOR FUNCIONA

A máscara só deveria mudar onde caem os 256 pares

1

Escolher pares

Sorteia 256 pares de pixels na imagem do drone, conforme uma máscara de probabilidade

2

Bits do drone

Para cada par compara 2 canais Lab: 512 bits

3

Bits no mapa

Projeta os mesmos pares no mapa, na pose de cada partícula (x, y, z, yaw)

4

Peso

Distância de Hamming → verossimilhança gaussiana → peso da partícula

abBRIEF (α = 0)

Máscara = gaussiana centrada. Pares concentrados no centro da imagem.

λ-BRIEF (α > 0)

Máscara = α · (−NDVI equalizado) + (1 − α) · gaussiana. Pares evitam a vegetação.

PROBLEMA 1

ab e λ rodavam por funções diferentes

```
phir2frameworkv2/ADM_036_30x_bateria_testes_paper.sh
:38 abBRIEF … -msc 0 … -pdm 1 …
:40 λ (a…) … -msc 1 … -pdm 51 … (idem :42-:58)
```

abBRIEF · -msc 0

`Mcl.cpp:712 prepareWeighting(z)`
`Mcl.cpp:1854 updateDroneDescriptor(drone)`
`BriefHeuristic.cpp:1132`

Função antiga do abBRIEF. Gera os pares por conta própria e não lê o **-pdm**.

λ-BRIEF · -msc 1

`Mcl.cpp:710 prepareWeighting(z, bandas…)`
`Mcl.cpp:1814 updateDroneDescriptor(…, NIR, RED…)`
`BriefHeuristic.cpp:70 → pixelDistribution, case 51 (:530)`

Roleta sobre a máscara gaussiana + NDVI, usada para α > 0.

Consequência: o α = 0 do artigo não é o mesmo código do α > 0. Mudar de ab para λ trocava a função inteira de amostragem, não só a máscara.

PROBLEMA 2

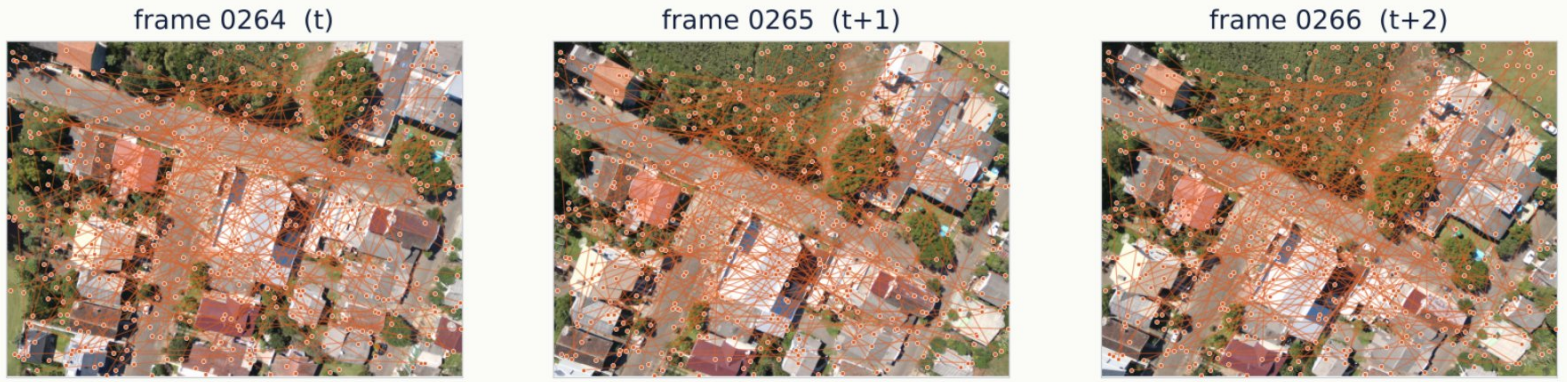
O abBRIEF usava sempre os mesmos 256 pares

BriefHeuristic.cpp:1132-1154

```
void updateDroneDescriptor(Mat& drone) {
cv::RNG rng; // :1145, semente padrão
for (x = 0; x < 256; x++)
do { pairs[x][0] = rng.gaussian(…) … }
while (!pointIn(pairs[x][0], drone));
```

cv::RNG rng; sem argumento nasce sempre com a mesma semente (0xFFFFFFFF), e a variável é recriada a cada chamada.

A função roda a cada frame, mas devolve sempre os **mesmos 256 pares**, em toda repetição, mapa e voo. No BRIEF original o padrão fixo é intencional; o problema é que o **λ re-sorteava** e o ab não.



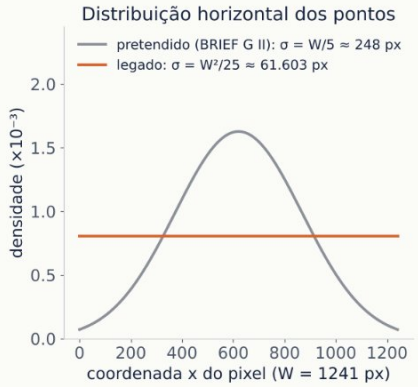
PROBLEMA 2

A variância foi usada como desvio-padrão

```
BriefHeuristic.cpp:1149
rng.gaussian(1.0/25.0*pow(drone.cols,2))
+ drone.cols/2
// cv::RNG::gaussian(σ) recebe o DESVIO-PADRÃO
// BRIEF G II: σ² = S²/25 → σ = S/5
```

Com W = 1241 px, o código pede **σ ≈ 61.600 px**, quando deveria pedir **σ ≈ 248 px**.

O laço **do … while (!pointIn)** descarta o que cai fora da imagem. O que sobra é praticamente **uniforme**, sem nenhuma concentração no centro.





O LAMBDA DO LEGADO

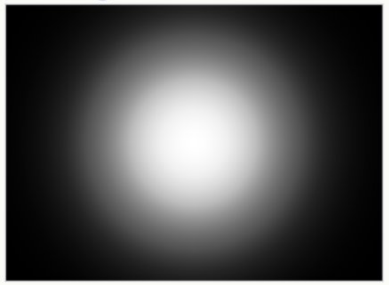
O λ: centro, NDVI e pares novos por frame

`gaussianMask :99–116 · NDVI → equalizeHist :565–620 · F = α·ϑ + (1-α)·N :621 · srand(time·tick) :539 + roleta :650–690`

imagem RGB



N: gaussiana central



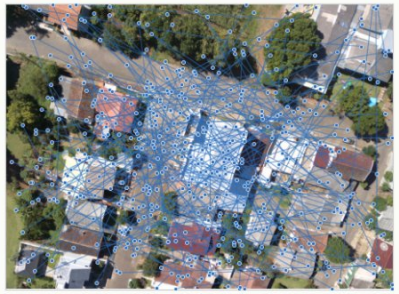
ϑ: −NDVI equalizado



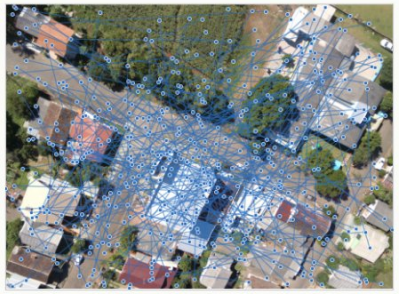
F = α·ϑ + (1−α)·N (α = 0,5)



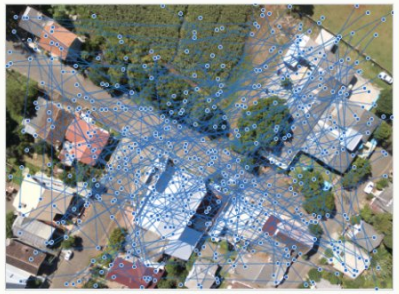
frame 0264



frame 0265



frame 0266



O QUE O ARTIGO COMPARAVA

Três coisas mudavam ao mesmo tempo

Aspecto	abBRIEF no artigo	λ-BRIEF no artigo
Função	updateDroneDescriptor(drone), :1132	pixelDistribution, case 51, :530
Onde caem os pares	Uniformes na imagem toda	Centro (gaussiana) e fora da vegetação
Quando são sorteados	Uma vez só: padrão fixo	A cada frame
Semente	Fixa (cv::RNG padrão)	Relógio (srand(time·tick))
Entre repetições e mapas	Idêntico	Diferente

⚠ O ganho que o artigo atribuiu à vegetação somava três efeitos: **centralizar** os pares, **re-sortear** a cada frame e **usar o NDVI**.

POR QUE DISTORCE

Cada diferença favorece o λ, com ou sem vegetação

Pares na borda

Na borda há mais distorção da lente e da inclinação do drone (o legado rodava sem compensação, -corrpxpos 0) e menos sobreposição com o mapa.

Um par fora do mapa empilha 3 bits onde cabem 2 (BriefHeuristic.cpp:1407–1409) e desalinha o resto do descritor.

Padrão fixo

Se os 256 pares caírem mal (vegetação, sombra, borda), caem mal em todos os frames.

Re-sortear a cada frame funciona como uma média de vários descritores ao longo do voo: um falso positivo de um padrão não se repete no próximo.

Mesmo padrão nas 30

O padrão do ab é o mesmo nas 30 repetições, nos 5 mapas e nos 4 voos.

A variação entre execuções do ab vinha só do filtro; a do λ, do filtro e do descritor.

COMO FAZEMOS AGORA

Um caminho só: ab e λ diferem apenas no α

```
apps/run_localization.cpp, a cada frame
:177 PixelSampler sampler(seed);
:273 gauss = generateGaussianMask(…);
:276 F = combineMasks(ndvi, gauss,
config.alpha);
:284 pairs = sampler.sampleRoulette(F);
```

- **α = 0** é só a gaussiana: o abBRIEF.
- Mesma roleta e re-sorteio nos dois.
- σ = W/5 e H/5 (mask_generator.cpp:13).
- Mesma seed para ab e λ: **pareado**.



COMO AS 30 REPETIÇÕES RODAVAM

As 30 repetições rodam em cadeia

```
./PhiR2Framework … -o "$resultPath" $count $(($count+$numTests-1)) # 0 a 29: um processo
DroneRobot::run :776-783 fim da trajetória → current++; reinitialize(); → MCL::restart (:507)
```

Repetição 1

Construtor do MCL. Semente do relógio, z sorteado em [0,1; 3]

→

Repetição 2

restart(): nuvem x, y, yaw da semente padrão, z = z final da 1

→

Repetição 3

A mesma nuvem x, y, yaw da 2; z = z final da 2

→

… até a 30

Cada uma herda a escala da anterior

Só a repetição 1 é uma localização global “fria”. As outras 29 começam de um estado que depende da anterior.

O QUE O RESTART FAZIA

MCL::restart sem semente e com a escala no campo errado

Construtor · repetição 1 (Mcl.cpp)

```
:57 seed = system_clock::now()…;
:62 default_random_engine generator(seed);
:84 particles[i].p.z = randomS(generator);
```

MCL::restart · repetições 2 a 30

```
:144 default_random_engine generator;
// sem semente: sempre a mesma sequência
:165 particles[i].s = randomS(generator);
// .s não é usado; p.z fica como estava
```

Nuvem inicial idêntica

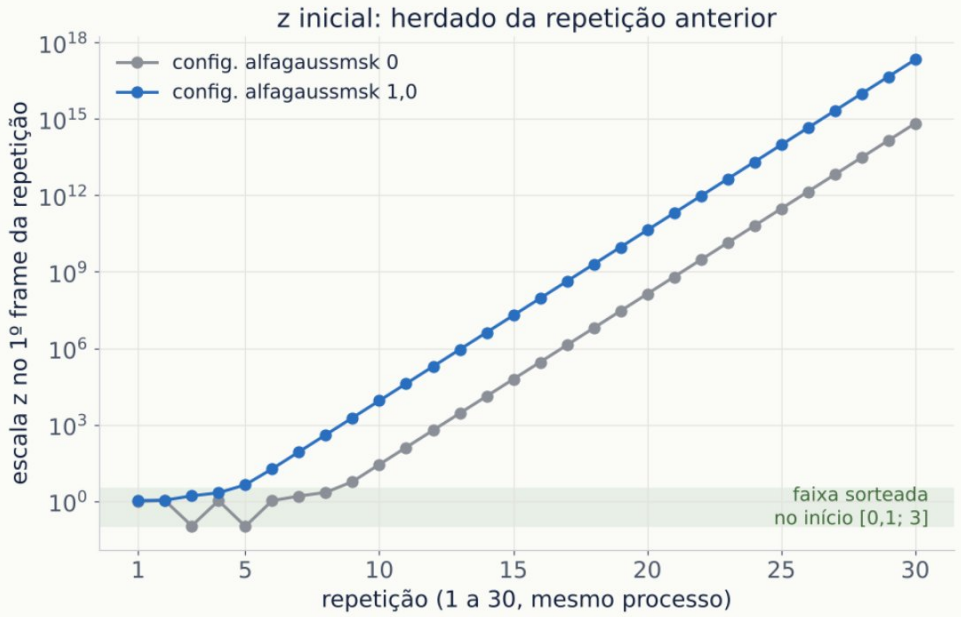
x, y e yaw saem sempre da mesma sequência: as repetições 2 a 30 começam com a mesma nuvem, em qualquer mapa e descritor.

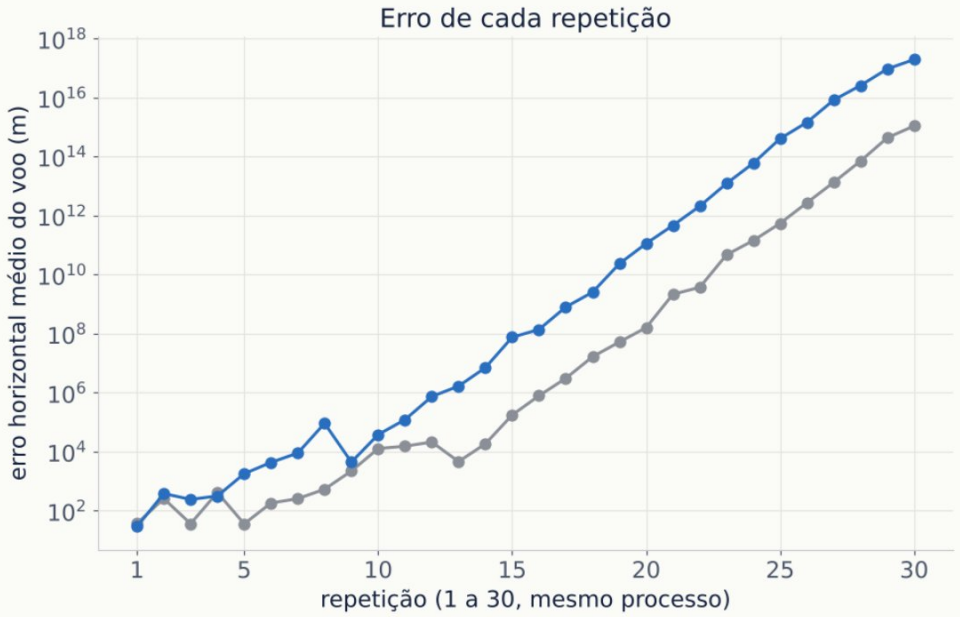
Escala herdada

O filtro usa p.z (Mcl.cpp:1176–1185). Como restart grava em .s, cada repetição começa com o z do fim da anterior.

EVIDÊNCIA NOS LOGS DE 2019

A escala passa de uma repetição à outra





O z do 1º frame da repetição k + 1 é o z do último frame da repetição k: **1,05 → 1,05**, **0,10 → 0,11**.

Sem nada que reinicie a escala, ela se multiplica pela escala da VO a cada voo: ×4 a ×5 por repetição a partir da 6ª, neste lote.

IMPACTO NO TESTE DO ARTIGO

A média de 30 virava a média de uma cadeia

n efetivo ≪ 30

Só a 1ª repetição é independente. As outras 29 dependem dela, então a curva média reflete muito o que aconteceu na 1ª.

Um prior de escala escondido

Se a 1ª convergiu, as próximas já começam com a altura certa e ficam mais fáceis que uma localização global real.

Ou o erro se propaga

Se a 1ª terminou com escala errada, as seguintes herdam o erro e podem divergir em cascata.

ab e λ em cadeias diferentes

Cada método e mapa era outro processo. Um desfecho diferente na 1ª repetição mudava as 30 curvas e ampliava a diferença.

COMO FAZEMOS AGORA

Uma execução por seed, do zero, pareada

```
lambda_brief_modern
lambda_brief_loc … --seed N # 1 processo
particle_filter.cpp:14-20 rng_.seed(seed)
particle_filter.cpp:23-41 x, y, z, yaw sorteados
run_localization.cpp:177 PixelSampler(seed)
…/seed_N/cmd.txt comando gravado
```

Cada execução sorteia a nuvem inteira, **inclusive z**, a partir da própria seed. Nada passa de uma execução para outra.

seed	1	2	…
ab	nuvem A	nuvem B	…
λ	nuvem A	nuvem B	…

Mesma seed, mesma nuvem inicial: a única diferença entre ab e λ é a máscara. Isso permite o teste pareado (Wilcoxon).

16 · repetições · benchmark_runner.py, paper_comparison_seeds.py

COM O MESMO BASELINE E SEEDS INDEPENDENTES

O λ converge mais vezes; quando converge, erra igual

Mapa (ADM2)	Conv. ab	Conv. λ	MAE ab	MAE λ	Mediana ab / λ
12/09/2018	53%	80%	177 m	106 m	15,0 / 15,1 m
17/08/2018	77%	83%	102 m	93 m	10,3 / 10,7 m
17/12/2018	63%	73%	151 m	132 m	11,8 / 11,7 m
27/02/2017	73%	87%	136 m	77 m	16,0 / 15,6 m

Motor novo, regime do artigo (sem altímetro, odometria do legado), α = 0,5, **30 seeds independentes** por mapa.

Com barômetro, injeção e duas fases, os dois convergem em 5/5 seeds em todos os mapas, e a diferença quase some.

PRÓXIMOS PASSOS

Medir o λ pelo que ele melhora: a convergência

1

Protocolo do artigo no motor novo

Sem altímetro, α = 1, 5 a 30 seeds, os 4 voos: taxa de convergência por mapa.

2

abBRIEF legado como opção

Pares uniformes e fixos no motor novo, para medir quanto do ganho do artigo vinha do baseline.

3

Métricas separadas

Taxa de convergência, tempo até convergir e erro depois de convergir, em vez de só o MAE médio.